

Report TWEB+IUM 2024-2025

Stoica Stefanut Cosmin

Frontend

Solution

È stato implementato il frontend utilizzando un template React + Vite. La scelta di React è stata guidata dalle esperienze maturate durante il percorso di studi e dalla consapevolezza che si tratti di una libreria JavaScript molto valida per lo sviluppo web moderno, in grado di offrire una struttura solida e flessibile. Vite è stato scelto come strumento di build per le sue elevate prestazioni in fase di sviluppo, che hanno permesso un ciclo di lavoro rapido ed efficace.

Il sito è stato progettato con un numero limitato di route, ritenute sufficienti a coprire tutte le funzionalità necessarie per un'applicazione di questo tipo. È stata realizzata una homepage ben strutturata, contenente informazioni utili e dati di interesse per l'utente. Inoltre, ogni film dispone di una pagina dedicata, così come ogni attore, entrambe organizzate in modo chiaro e facilmente navigabile.

Per la sezione dedicata agli attori è stato integrato un dataset pubblico disponibile su Hugging Face, che ha permesso di visualizzare informazioni personali e immagini degli attori. A causa delle dimensioni non particolarmente estese del dataset, il numero di attori recuperabili è stato limitato a circa 10.000 elementi, che rappresentano comunque i profili più noti.

È stata implementata una strategia di caching lato frontend, ispirata alla libreria open-source TanStack Query, con l'obiettivo di migliorare le prestazioni e ridurre il numero di richieste duplicate verso le API. In particolare, è stata realizzata una cache in-memory che consente di gestire la validità temporale dei dati e di controllare quando un fetch debba essere ripetuto.

La funzionalità di chat è stata sviluppata utilizzando Socket.IO, consentendo agli utenti di inviare e ricevere messaggi in tempo reale e di visualizzare immediatamente i messaggi scritti da altri utenti collegati.

Nel complesso, la soluzione frontend realizzata risponde agli obiettivi del progetto e offre un'interfaccia semplice, intuitiva e facilmente estendibile.

Issues

Una delle principali difficoltà riscontrate riguarda la progettazione del sistema di caching. È stato necessario realizzare un meccanismo sufficientemente robusto da gestire in modo efficiente tutte le richieste API, migliorando il flusso dei dati tra frontend e backend.

Per affrontare questa problematica sono stati presi come riferimento i concetti e le strategie adottate dalla libreria TanStack Query, la cui documentazione è interamente accessibile.

Inoltre, per valutare le possibili soluzioni architetturali, è stato utilizzato anche il supporto di un LLM (ChatGPT), principalmente come strumento di confronto e supporto decisionale.

Requirements

Il design adottato risulta complessivamente robusto, pur lasciando spazio a possibili miglioramenti futuri, in particolare per quanto riguarda la sicurezza e la gestione avanzata degli errori. Il sito risulta semplice da navigare e intuitivo, rispettando i principi di progettazione orientata all'utente (HCI-driven).

Dal punto di vista architetturale, l'utilizzo combinato di React e Vite consente di ottenere una struttura scalabile e facilmente manutenibile. Le richieste API sono documentate tramite Swagger e restituiscono correttamente i dati. Grazie al sistema di caching implementato, il carico sulle API risulta ridotto e la gestione delle richieste più efficiente.

Tutto il codice è commentato utilizzando JSDoc (template *better-docs*) e la documentazione viene generata tramite il comando `npm run docs`, con output nella directory `$root/docs`.

Sono state inoltre utilizzate le librerie `flag-icons` e `i18n-iso-countries`, che hanno facilitato il parsing dei nomi dei paesi e la loro associazione alle rispettive bandiere.

Limitations

Una possibile limitazione riguarda la validazione dei dati ricevuti dalle API, che è presente ma potrebbe essere resa più completa e rigorosa. Dal punto di vista architetturale, la suddivisione dei moduli e la struttura del progetto risultano ben organizzate e consentono di estendere facilmente la soluzione per soddisfare futuri requisiti.

Express servers

Solution

Sono stati realizzati due gateway backend in Node.js utilizzando Express 5, con il compito di gestire tutte le rotte REST esposte sotto `/api` e di fungere da ponte verso il backend Spring Boot con PostgreSQL e verso MongoDB per la gestione della chat.

È stato utilizzato Axios con un client preconfigurato per comunicare con Spring Boot, affiancato da una funzione helper che consente di riutilizzare gli stessi handler anche nel caso in cui i percorsi esposti dal backend differiscano. Per i contenuti statici, come poster dei film e immagini degli attori, le URL vengono normalizzate lato gateway per fornire link assoluti compatibili con l'SPA.

È stata integrata la documentazione delle API tramite Swagger/OpenAPI, accessibile tramite gli endpoint /docs e /openapi.json, garantendo una documentazione sempre aggiornata. Il CORS è stato configurato con credenziali, limitando l'origine allo SPA, e tutte le risposte seguono un formato JSON uniforme { ok, data, error }, con un gestore centrale degli errori.

Per ridurre il carico sul sistema è stata introdotta una cache in-memory basata su NodeCache, utilizzata in particolare per le operazioni di ricerca. La chat è stata gestita tramite MongoDB con l'ausilio di Mongoose, che permette la definizione degli schemi, la validazione dei dati e la creazione automatica di collezioni e indici.

Le funzionalità di chat sono disponibili tramite Socket.IO. Il main gateway inoltra al middle server le room global o movie:<id>, dunque gli utenti possono accedere a una stanza globale oppure a stanze specifiche associate a un film, inviare e ricevere messaggi in tempo reale e caricare lo storico tramite paginazione a cursore. E' stata aggiunta anche la persistenza dei messaggi dentro mongoDB.

Issues

Una delle principali sfide è stata l'integrazione efficace della pipeline tra frontend, gateway Express e backend Spring Boot. La definizione e l'organizzazione delle API ha richiesto particolare attenzione, in quanto rappresenta uno degli aspetti centrali dell'architettura client-server. Le scelte effettuate sono state valutate rispetto ai requisiti del progetto e ritenute adeguate.

Requirements

Il server soddisfa i requisiti richiesti dall'assegnazione: è stato utilizzato Express per la gestione delle API REST, Socket.IO per la chat in tempo reale e MongoDB per la persistenza delle review e chat. Le stanze di chat sono organizzate per film oppure in una stanza globale, introdotta come funzionalità aggiuntiva per migliorare l'esperienza utente.

Limitations

Il gateway è stato progettato tenendo conto della possibilità di upstream lenti o non disponibili, introducendo timeout e logging di base. Tuttavia, mancano meccanismi più avanzati come circuit breaker, backoff o bilanciamento del carico. L'architettura è estendibile e consente di aggiungere facilmente nuove rotte o middleware, ma risulta ancora carente dal punto di vista della sicurezza e dell'osservabilità globale di errori upstream più ricco e una gestione più robusta dei picchi (pool/queue). In sintesi il progetto è leggero e modificabile, ma va completato con sicurezza e monitoraggio per scenari eccezionali.

Server Springboot

Solution

Il backend applicativo è stato sviluppato utilizzando Spring Boot, esponendo cinque principali categorie di API: film, attori, Oscar, recensioni e ricerca. Spring Boot è stato scelto per la completezza del suo stack, che include supporto nativo a JPA, validazione dei dati e dependency injection, oltre a un'ottima integrazione con PostgreSQL.

I dati sono stati inizialmente forniti sotto forma di file CSV e successivamente trasformati e normalizzati in tabelle relazionali. Sono state create tabelle dedicate per la gestione delle relazioni many-to-many e dei ruoli associati a film e persone. Le immagini degli attori, ottenute da un dataset esterno, sono state scaricate e correttamente associate agli identificativi presenti nel database.

Sono stati utilizzati DTO per definire in modo chiaro i payload restituiti dalle API, separando la logica di presentazione dalla logica di persistenza. Le operazioni sui dati sono state incapsulate in servizi JPA, garantendo una struttura ordinata e facilmente estendibile.

Particolare attenzione è stata posta alla documentazione del codice e delle API. Tutte le classi principali, i controller e i servizi sono stati commentati utilizzando Javadoc, rendendo il codice più leggibile e comprensibile. Inoltre, è stata integrata la documentazione OpenAPI/Swagger, che descrive automaticamente tutti gli endpoint disponibili, i parametri richiesti e le risposte restituite. Questo consente una consultazione immediata delle API e facilita sia lo sviluppo che la manutenzione del progetto.

Per quanto riguarda la ricerca, sono stati creati indici specifici in PostgreSQL per migliorare le prestazioni delle query e limitato il numero di risultati restituiti, evitando rallentamenti in fase di esecuzione.

Issues

La principale difficoltà è stata la trasformazione dei dati provenienti dai file CSV in un modello relazionale consistente. Questo processo ha richiesto operazioni di pulizia, normalizzazione e creazione di relazioni corrette tra le entità.

Un'ulteriore complessità è stata l'implementazione di funzionalità di ricerca efficienti, che ha richiesto la creazione di indici ad hoc e una gestione attenta della paginazione per evitare query troppo costose.

Requirements

Le API richieste dall'assegnazione sono tutte presenti e correttamente implementate: film, attori, Oscar, recensioni e ricerca. Ogni endpoint espone DTO ben definiti e restituisce risultati ordinati e limitati. La ricerca è indicizzata e paginata.

La documentazione delle API è disponibile tramite OpenAPI/Swagger, mentre il codice è interamente commentato con Javadoc, soddisfacendo i requisiti di chiarezza, manutenibilità e documentazione richiesti dal progetto.

Limitations

In caso di carichi elevati o di rallentamenti del database, sarebbero necessari ulteriori meccanismi di caching e monitoraggio. Inoltre, la gestione globale delle eccezioni potrebbe essere arricchita e resa più dettagliata.

L'architettura rimane comunque estendibile e ben organizzata, consentendo l'aggiunta di nuovi endpoint, nuovi indici o funzionalità avanzate senza modifiche strutturali rilevanti.

Bibliography

1. <https://huggingface.co/datasets/ashraq/tmdb-celeb-10k>
2. <https://tanstack.com/query/latest/docs/framework/react/overview>